



COM3021 – Data Science at Scale

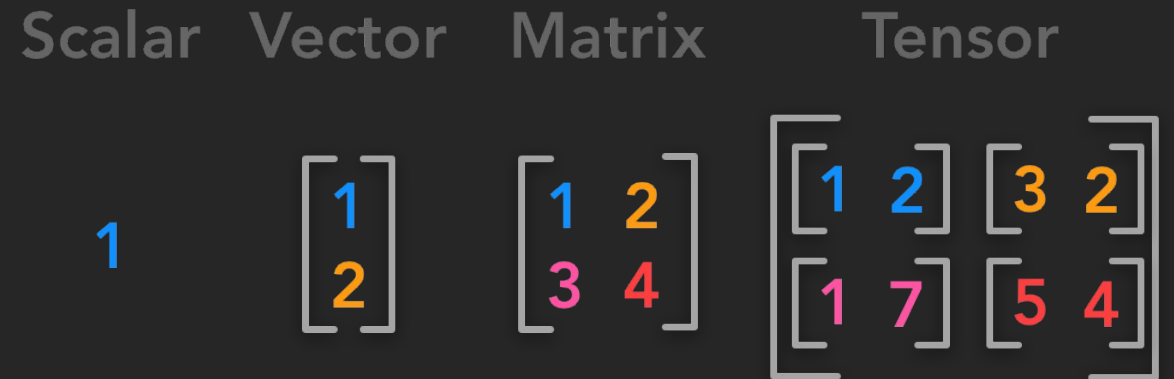
Tensors

DR YANDA MENG



Introduction

- Data is frequently represented as tensors
 - *0-order: scalar*
 - *1-order: vector*
 - *2-order: matrix*
 - *3-order: tensor*



Introduction

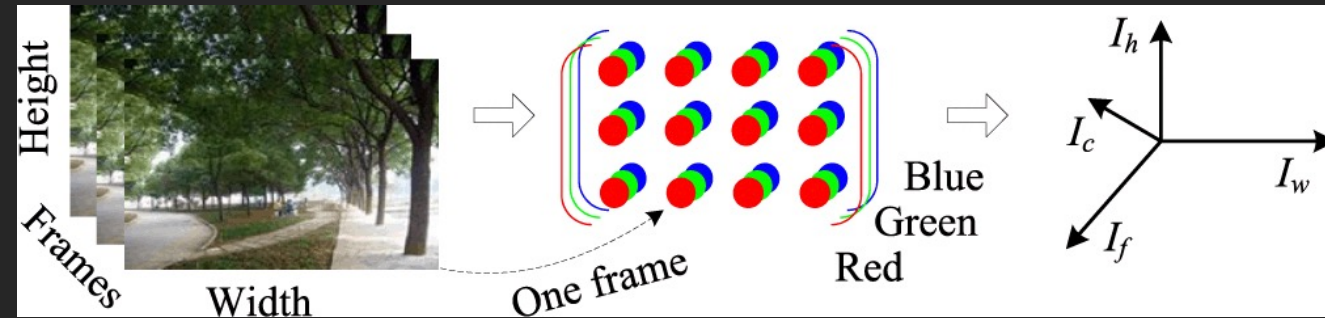
- Data is frequently represented as tensors

Vector data: features

Timeseries: features $\times$ timestamps

Image: width $\times$ height $\times$ channels (e.g. RGB)

Video: frames $\times$ width $\times$ height $\times$ channels



Sparse tensors

- A sparse tensor is one in which most of the elements are 0

If the number of non-zero elements in an $n \times n$ tensor is approximately in the order of n

- A dense tensor is one in which most elements are non-zero

Sparsity: the proportion of zero-valued elements in a tensor

1.0	0	5.0	0	0	0	0	0
0	3.0	0	0	0	0	11.0	0
0	0	0	0	9.0	0	0	0
0	0	6.0	0	0	0	0	0
0	0	0	7.0	0	0	0	0
2.0	0	0	0	0	10.0	0	0
0	0	0	8.0	0	0	0	0
0	4.0	0	0	0	0	0	12.0

Sparse tensors

- Real-world data is frequently very sparse

Social graph

Fault detection sensors

- Representing large matrices ($n \times n$ with $n \sim 10^6$) might be prohibitive.

64 bits values $\rightarrow$ 58GB

- When tensors are sparse, we can find other ways to represent data more efficiently
-

Sparse tensor

- A sparse representation in which we store only the non-zero values

Dictionary of keys (row, column): value

(0,0): 1.0

(0,2): 5.0

(1,1): 3.0

(1,5): 11.0

...

1.0	0	5.0	0	0	0	0	0
0	3.0	0	0	0	0	11.0	0
0	0	0	0	9.0	0	0	0
0	0	6.0	0	0	0	0	0
0	0	0	7.0	0	0	0	0
2.0	0	0	0	0	10.0	0	0
0	0	0	8.0	0	0	0	0
0	4.0	0	0	0	0	0	12.0

Sparse tensor

- A sparse representation in which we store only the non-zero values

Coordinate list

List of (row, column, value tuples

(0,0,1.0), (0, 2,5.0), (1,1,3.0)...

1.0	0	5.0	0	0	0	0	0
0	3.0	0	0	0	0	11.0	0
0	0	0	0	9.0	0	0	0
0	0	6.0	0	0	0	0	0
0	0	0	7.0	0	0	0	0
2.0	0	0	0	0	10.0	0	0
0	0	0	8.0	0	0	0	0
0	4.0	0	0	0	0	0	12.0

Sparse tensor

- Compressed sparse row (CSR)

Three arrays:

V: Non-zero values

COL_INDEX: Column index of the non-zero elements

ROW_INDEX (length $m+1$): encodes the index in V and COL_INDEX where the given row starts.

To reconstruct a row:

$row_start = ROW_INDEX[row]$

$row_end = ROW_INDEX[row + 1]$

Make the slices

$V[row_start:row_end]$ values

$COL_INDEX[row_start:row_end]$ column indexes

Sparse tensor

- Compressed sparse row (CSR)

$$V = [5 \ 8 \ 3 \ 6]$$

$$COL_INDEX = [0 \ 1 \ 2 \ 1]$$

$$ROW_INDEX = [0 \ 1 \ 2 \ 3 \ 4]$$

$$\begin{pmatrix} 5 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 6 & 0 & 0 \end{pmatrix}$$

Sparse tensor

- Compressed sparse row (CSR)

Exercise: Build the third row from the following CSR

$V = [10 \ 20 \ 30 \ 40 \ 50 \ 60 \ 70 \ 80]$

$COL_INDEX = [0 \ 1 \ 1 \ 3 \ 2 \ 3 \ 4 \ 5]$

$ROW_INDEX = [0 \ 2 \ 4 \ 7 \ 8]$

To reconstruct a row:

$row_start = ROW_INDEX[row]$

$row_end = ROW_INDEX[row + 1]$

Make the slices

$V[row_start:row_end]$ values

$COL_INDEX[row_start:row_end]$ column indexes

Sparse tensor

- Compressed sparse row (CSR)

Exercise: build the CSR representation

$$\begin{pmatrix} 10 & 20 & 0 & 0 & 0 & 0 \\ 0 & 30 & 0 & 40 & 0 & 0 \\ 0 & 0 & 50 & 60 & 70 & 0 \\ 0 & 0 & 0 & 0 & 0 & 80 \end{pmatrix}$$